PROJETO DE ESTRUTURAS DISCRETAS | INF1631 - 3WB

Circuitos de Euler em grafos aleatórios

Relatório de implementação e resultados

Grupo F | 31 de outubro de 2026

Dante Honorato Matrícula: 2321406
Breno Soares Matrícula: 2320363

1 Objetivo e parâmetros

O projeto investiga se um robô consegue inspecionar todas as ruas de uma rede, percorrendo cada rua exatamente uma vez e retornando ao cruzamento inicial. A rede é representada por um grafo simples não direcionado: os vértices são os cruzamentos e as arestas são as ruas. A missão corresponde à existência de um circuito de Euler.

Foram realizados dois experimentos: gerar grafos até encontrar o primeiro Euleriano e classificar uma amostra de 1.000 grafos. Os parâmetros oficiais do grupo F foram mantidos em ambos:

Parâmetro	Valor	Significado
n	10	Número de vértices
p	0,30	Probabilidade de inclusão de cada aresta
Seed	641	Semente do gerador pseudoaleatório

O conjunto de vértices é $V = \{1, 2, \ldots, 10\}$. Para cada um dos $\binom{10}{2} = 45$ pares distintos, o programa faz um sorteio e inclui a aresta quando o valor obtido é menor que p. Cada par é considerado uma única vez, impedindo laços e arestas duplicadas. O programa não constrói deliberadamente um grafo Euleriano.

2 Fundamentação e reprodutibilidade

Para um grafo com arestas, a existência de um circuito de Euler exige que todos os vértices de grau não nulo pertençam à mesma componente conexa e tenham grau par. A paridade permite que cada chegada a um vértice seja associada a uma saída. A conectividade garante que todas as arestas possam ser alcançadas em um único percurso.

Vértices isolados não impedem a inspeção das ruas, pois não possuem arestas a percorrer. Para o caso sem nenhuma aresta, adotou-se explicitamente a convenção de não considerar a rede uma solução de inspeção. Esse caso é tratado no código e nos testes.

Cada experimento começa com `random.Random(641)`. Dentro de cada experimento, o estado do gerador avança normalmente, sem reinicialização a cada grafo. O segundo experimento reinicia a mesma sequência da busca, permitindo reproduzir os resultados com o mesmo código e ambiente.

3 Descrição da implementação

A implementação foi escrita em Python e utiliza apenas a biblioteca padrão: `random` para os sorteios, `time` para a medição e `argparse` para receber os parâmetros. Os algoritmos de grafos foram implementados diretamente, sem bibliotecas que determinem a existência de um circuito ou o construam automaticamente.

O grafo é armazenado em um dicionário que associa cada vértice a um conjunto de vizinhos. Ao incluir $\{u, v\}$, o programa adiciona v aos vizinhos de u e u aos vizinhos de v, preservando a representação não direcionada.

Função	Procedimento implementado
`gerar_grafo(n,p)`	Percorre os pares $u < v$ e sorteia a presença de cada aresta. Recebe opcionalmente o objeto gerador.
`graus(G)`	Retorna a quantidade de vizinhos de cada vértice.
`conexo(G)`	Usa uma busca em profundidade com pilha e compara os vértices visitados com os vértices de grau não nulo.
`euleriano(G)`	Exige pelo menos uma aresta, conectividade dos vértices ativos e paridade de todos os graus.
`circuito_euler(G)`	Executa o algoritmo de Hierholzer sobre uma cópia das adjacências. Retorna `None` se não houver solução.
`valida_circuito(G,C)`	Verifica fechamento, número de passos, existência das arestas, ausência de repetição e cobertura de todas as arestas.

3.1 Construção do circuito

O algoritmo de Hierholzer começa no menor vértice que possui arestas. Enquanto há uma aresta disponível no vértice do topo da pilha, o programa escolhe o menor vizinho, remove a aresta nos dois sentidos e avança. Quando não há arestas disponíveis, retira o vértice da pilha e o acrescenta à lista de saída. A inversão dessa lista produz o circuito.

A escolha do menor vizinho torna a apresentação do percurso determinística. A remoção ocorre em uma cópia do grafo, preservando o original para a exibição e para a validação posterior.

3.2 Validação automática

O validador representa cada aresta não direcionada pela dupla $(\min(u, v), \max(u, v))$. Ele exige que o circuito seja fechado e contenha $|E| + 1$ vértices, isto é, $|E|$ deslocamentos. Cada deslocamento deve corresponder a uma aresta existente e ainda não utilizada. Ao final, o conjunto percorrido deve ser exatamente igual ao conjunto de arestas do grafo. Assim, a verificação não depende somente da inspeção visual.

4 Primeiro experimento: busca por um grafo Euleriano

A busca encontrou o primeiro grafo Euleriano na **tentativa 818**. Os 817 grafos anteriores foram rejeitados pelo critério implementado. O grafo encontrado possui 10 vértices e 10 arestas, listadas abaixo:

$$E = \{\,(2,6),(2,10),(3,5),(3,6),(3,8),\\ (3,10),(5,8),(6,8),(6,10),(8,10)\,\}.$$

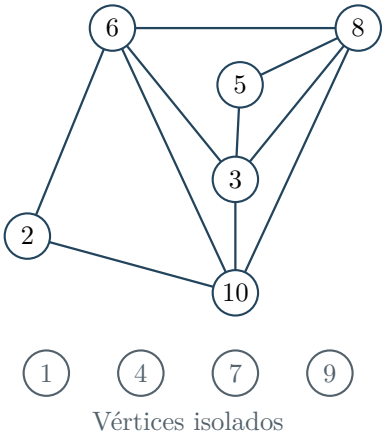


Figura 1: grafo encontrado na tentativa 818.

Vértice	Grau
1	0
2	2
3	4
4	0
5	2
6	4
7	0
8	4
9	0
10	4

Os vértices ativos são $\{2,3,5,6,8,10\}$ e pertencem à mesma componente conexa. Seus graus são pares. A soma dos graus é $20 = 2|E|$, consistente com as 10 arestas. Os vértices 1, 4, 7 e 9 são isolados e não precisam ser visitados para percorrer todas as ruas.

4.1 Circuito encontrado e validado

$$\boxed{2 \to 6 \to 3 \to 5 \to 8 \to 3 \to 10 \to 6 \to 8 \to 10 \to 2}$$

O percurso começa e termina no vértice 2. Seus 10 deslocamentos usam exatamente as 10 arestas do grafo, sem repetição. O validador retornou **OK**. A repetição de vértices no percurso é permitida; a restrição de uso único se aplica às arestas.

Embora o grafo completo tenha vértices isolados, sua componente com arestas é conexa. Portanto, ele atende ao critério de conectividade solicitado e permite realizar a missão de inspeção.

5 Segundo experimento: amostra de 1.000 grafos

O gerador foi reinicializado com a *seed* 641 e foram gerados exatamente 1.000 grafos com $n = 10$ e $p = 0{,}30$. Não houve alteração dos parâmetros nem reinicialização entre os grafos da amostra.

A medição com `time.perf_counter()` envolve somente a criação da lista dos 1.000 grafos. O teste de existência de circuito de Euler é executado depois do encerramento da medição. Assim, o tempo informado corresponde à geração e ao armazenamento dos grafos, sem incluir sua classificação ou a impressão dos resultados.

Medida	Resultado
Grafos gerados	1.000
Grafos Eulerianos	1
Grafos não Eulerianos	999
Proporção de Eulerianos	0,001
Percentual de Eulerianos	0,1%
Tempo de geração observado	0,011571 s

A proporção foi calculada conforme o enunciado:

$$\widehat{p}_{\text{Euler}} = \frac{\text{número de grafos Eulerianos}}{1000} = \frac{1}{1000} = 0{,}001 = 0{,}1\%.$$

O tempo acima foi observado em uma execução em 31/10/2026, com Python 3.12.8, macOS 26.5 e arquitetura arm64. É uma medição de execução, não uma constante do algoritmo: carga do sistema e equipamento podem alterá-la. As contagens, o grafo e o circuito são reproduzíveis com a mesma implementação e o mesmo estado inicial do gerador.

5.1 Análise dos resultados

A ocorrência de apenas um Euleriano indica baixa frequência na amostra observada. O resultado é compatível com a necessidade de satisfazer simultaneamente a paridade dos graus e a conectividade dos vértices ativos. A proporção empírica não deve ser interpretada como a probabilidade exata do modelo, pois foi calculada a partir de uma amostra finita.

Os graus pares são necessários para sair de cada vértice após uma chegada e retornar ao ponto inicial. Entretanto, paridade sozinha não basta: duas componentes cíclicas separadas possuem todos os graus pares, mas não permitem percorrer todas as arestas em um único circuito. Da mesma forma, um grafo conexo com vértices de grau ímpar pode impedir o circuito, como ocorre no caminho P_4.

Como o segundo experimento reinicia a mesma sequência pseudoaleatória, o grafo da tentativa 818 reaparece na posição 818 da amostra. Ele é o único Euleriano entre os primeiros 1.000 grafos dessa sequência. Portanto, os dois resultados são consistentes, mas não representam duas amostras independentes entre si. Dentro da amostra de 1.000 grafos, cada grafo é produzido com novos sorteios, conforme o modelo solicitado.

6 Testes e execução

Os testes estão organizados em `test_grafos.py`, utilizando `unittest`. Os quatro casos obrigatórios foram executados com os seguintes resultados:

Caso	Resultado esperado	Resultado obtido
Ciclo C_4	Circuito válido.	Euleriano; circuito validado e grafo original preservado.
Caminho P_4	Ausência de circuito de Euler.	Rejeitado por possuir vértices de grau ímpar.
Duas componentes cíclicas	Ausência de circuito de Euler.	Rejeitadas pela falta de conectividade entre vértices ativos.
Grafo sem arestas	Tratamento explícito.	Não Euleriano pela convenção de inspeção; construção retorna `None`.

Também foram executados testes adicionais para vértices isolados, rejeição de percursos inválidos e geração reproduzível, incluindo os extremos $p = 0$ e $p = 1$ na função geradora. Esses casos unitários servem para testar a implementação; os dois experimentos do grupo usam exclusivamente $n = 10$, $p = 0{,}30$ e *seed* 641. O resultado da suíte foi **7 testes executados e aprovados**.

6.1 Reprodução

Não são necessárias bibliotecas externas para executar o programa. Na pasta do projeto, os comandos são:

```
python3 -X utf8 main.py 0.30 641
python3 -m unittest -v test_grafos
```

Em ambientes que disponibilizam o interpretador como `python`, basta substituir `python3`. O arquivo `README.md` contém as instruções de execução e de compilação deste relatório.

7 Conclusão

Foi implementado um programa que gera redes aleatórias, aplica os critérios matemáticos de existência de circuito de Euler, constrói um percurso por Hierholzer e o valida automaticamente. Com os parâmetros do grupo F, a primeira solução apareceu após 818 grafos. O circuito encontrado percorre as 10 arestas uma única vez e retorna ao ponto inicial.

Na amostra de 1.000 grafos, apenas um foi Euleriano, correspondendo a 0,1%. O resultado evidencia, nessa execução, a baixa frequência de redes que satisfazem simultaneamente as condições de grau e conectividade. Os testes confirmaram os comportamentos esperados, incluindo o tratamento de vértices isolados e de grafos sem arestas.

Base do trabalho: enunciado *Projeto de Estruturas Discretas – Grafos*, INF1631 - 3WB, 2026, e tabela de parâmetros por grupo fornecida na disciplina. Implementação e testes: `main.py` e `test_grafos.py`.